

ubuntu20.04, ubuntu22.04 }. But you are not limited to the prebuilt images. You can use any custom Docker image with systemd and ssh daemons installed (see References).

For example, I used the Dockerfile shown below to create a customised Ubuntu 24.04 LTS local image to instantiate my container machines, when I tried server migration from the old prebuilt Ubuntu 22.04 LTS:

<Start of Dockerfile>

```
FROM ubuntu:24.04

ENV container docker

# Don't start any optional services
except for the few we need.
RUN find /etc/systemd/system \
    /lib/systemd/system \
    -path '*.wants/*' \
    -not -name '*journal*' \
    -not -name '*systemd-tmpfiles*' \
    -not -name '*systemd-user-
sessions*' \
    -exec rm \{\} \; && \
    apt-get update && \
    apt-get install -y --no-install-
recommends \
    dbus systemd openssh-server net-
tools iproute2 iputils-ping curl wget
vim-tiny sudo && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* && \
    true >/etc/machine-id && \
    true >/var/lib/dbus/machine-id && \
    systemctl set-default multi-user.
target && \
    systemctl mask \
```

```
dev-hugepages.mount \
sys-fs-fuse-connections.mount \
systemd-update-utmp.service \
systemd-tmpfiles-setup.service \
console-getty.service && \
# Seems dispatcher is not a part of
24.04LTS
# systemctl disable \
# networkd-dispatcher.service && \
sed -i -e 's/^AcceptEnv LANG
LC_*/$/#AcceptEnv LANG LC_*/' /etc/ssh/
sshd_config
# This container image doesn't have
locales installed. Disable forwarding
the
# user locale env variables or we get
warnings such as:
# bash: warning: setlocale: LC_ALL:
cannot change locale

#https://www.freedesktop.org/wiki/
Software/systemd/ContainerInterface/
STOPSIGNAL SIGRTMIN+3

EXPOSE 22

CMD ["/bin/bash"]
```

<End of Dockerfile>

To give you a better idea of the bootloose configuration, the configuration shown below uses the locally created ubuntu2404lts image (created using the *docker build -t ubuntu2404lts* command with the Dockerfile shown above) to build three container machines with host names kafka{0,1,2}. This will help to later install and configure Monit and Dockerd (need the *privileged* flag) in

these machines. Several ports now expose multiple services running in each container machine. You can also mix and match various kinds of container machines under the *machines* section in the same configuration.

<Start of Kafka Config>

```
cluster:
  name: cluster
  privateKey: cluster-key
machines:
- count: 3
  spec:
    image: ubuntu2404lts
    name: kafka%d
    networks:
    - cldinabox-demo
    portMappings:
    - containerPort: 22
    - containerPort: 2181
    - containerPort: 8080
    - containerPort: 9092
    - containerPort: 38080
    - containerPort: 52812
    - containerPort: 58080
    privileged: true
    volumes:
    - type: volume
      destination: /var/lib/docker
```

<End of Kafka Config>

First create the required network using the command *docker network create cldinabox-demo*. You can then bring up the container machines with the command *bootloose create* and dump their information with the command *bootloose show*, as shown in Figure 3.

```
INFO[0000] Creating machine: cluster-kafka0 ...
INFO[0000] Connecting cluster-kafka0 to the cldinabox-demo network...
INFO[0000] Creating machine: cluster-kafka1 ...
INFO[0000] Connecting cluster-kafka1 to the cldinabox-demo network...
INFO[0001] Creating machine: cluster-kafka2 ...
INFO[0001] Connecting cluster-kafka2 to the cldinabox-demo network...

NAME      IP      IMAGE      HOSTNAME  PORTS      STATE
cluster-kafka0  kafka0  ubuntu2404ltsfcs  0->{8080 32814},1->{9092 32813},2->{2181 32815},3->{22 32816},4->{38080 32812},5->{52812 32811},6->{58080 32810}
cluster-kafka1  kafka1  ubuntu2404ltsfcs  0->{8080 32821},1->{9092 32820},2->{2181 32822},3->{22 32823},4->{38080 32819},5->{52812 32818},6->{58080 32817}
cluster-kafka2  kafka2  ubuntu2404ltsfcs  0->{8080 32828},1->{9092 32827},2->{2181 32829},3->{22 32830},4->{38080 32826},5->{52812 32825},6->{58080 32824}
```

Figure 3: bootloose create and show